

ShopGuide Agent 完整实施方案 - 重构章节版

面向 Android 原生客户端、RAG 后端、低压力导购交互与三周落地的统一方案

项目组内部对齐文档

2026-05-24

Contents

文档阅读方式	3
1. 项目定位与核心判断	3
1.1 一句话定位	3
1.2 为什么不能做成普通 RAG 商品搜索	3
1.3 项目成功的判断标准	4
2. 用户体验设计：低压力、主推、可修正	4
2.1 低压力导购的核心原则	4
2.2 文本回复的风格	4
2.3 主推 1 个产品，而不是结果列表	4
2.4 主动反问的边界	5
3. 功能范围与优先级	5
3.1 P0：必须完成	5
3.2 P1：强烈建议完成	5
3.3 P2：有时间再做	5
4. 总系统架构	6
4.1 架构总览	6
4.2 两阶段 LLM + 确定性检索控制	6
5. 商品数据与专属知识库	7
5.1 商品 Schema	7
5.2 知识库构建	7
5.3 数据量建议	8
6. 后端 RAG、检索计划与排序逻辑	8
6.1 LLM 生成结构化检索计划	8
6.2 反选/排除约束	9
6.3 不采用固定线性加权排序	9
6.4 分层排序策略	10
Hard Filter	10
Eligibility Tier	10
In-tier Ranking	10
6.5 受控多跳 RAG	11
6.6 场景化组合推荐	11
7. 以商品为锚点的多轮导购	12
7.1 设计动机	12
7.2 核心流程	12
7.3 Product Focus Context	12
7.4 商品级追问示例	13

8. Android 客户端整体设计	13
8.1 Android 端职责	13
8.2 推荐技术栈	14
8.3 Android 工程目录	14
8.4 可借鉴的经典 Android 项目	15
9. Android 核心页面与组件	15
9.1 ChatScreen	15
9.2 商品卡片流式渲染	16
9.3 ProductDetailBottomSheet	16
9.4 BundleSection	16
9.5 CartScreen	17
10. Android 与后端协议	17
10.1 客户端发给后端	17
user_message	17
product_followup	17
cart_action	17
10.2 后端发给 Android	18
text_delta	18
product_item	18
replacement_product	18
bundle_item	18
audio_delta	19
10.3 REST 接口	19
11. Android Studio 与 Codex / Claude Code 协作	19
11.1 推荐协作模式	19
11.2 每次给 Codex 的固定模板	19
11.3 任务顺序	20
11.4 不要让 Codex 一次做全部	20
12. 三周实施计划与分工	20
12.1 Week 1: 端到端最小闭环	20
12.2 Week 2: 差异化能力	20
12.3 Week 3: 评测、稳定、答辩	21
12.4 两人分工	21
13. 评测、验收与答辩	21
13.1 必测 Query	21
13.2 评测指标	21
13.3 Android 验收清单	22
13.4 答辩表达	22
14. 风险控制	22
14.1 风险: 只推一个商品不稳	22
14.2 风险: LLM 漏掉否定约束	22
14.3 风险: 上下文污染	22
14.4 风险: TTS 卡顿	23
14.5 风险: Codex 改太多导致项目失控	23
15. 最终交付物	23
附录 A: 核心事件列表	23
附录 B: 最小 Demo 脚本	23
附录 C: 最终核心卖点	24

文档阅读方式

这份文档不是按照讨论过程排列，而是按照项目落地逻辑重新组织：

先明确产品目标

- > 再定义用户体验
- > 再拆系统架构
- > 再定后端 RAG 与排序逻辑
- > 再定 Android 端实现
- > 再定接口协议
- > 再定 Codex / Claude Code 执行任务
- > 最后给出三周排期、验收与风险控制

阅读建议：

- 项目负责人先读第 1-3 章，明确项目边界和卖点。
- 后端同学重点读第 4-6 章，明确 RAG、检索计划、排序、多跳和协议。
- Android 同学重点读第 7-10 章，明确客户端页面、组件、状态和任务顺序。
- 两个人一起读第 11-13 章，用于联调、评测、答辩和风险控制。

1. 项目定位与核心判断

1.1 一句话定位

ShopGuide Agent 是一个面向电商场景的 Android 原生 AI 导购系统。它不是自然语言商品搜索引擎，而是低压力、以商品为锚点、能理解用户购买动机并支持多轮修正的智能导购 Agent。

核心闭环：

用户用自然语言表达购物需求

- > 系统理解购买动机、硬约束和软偏好
- > LLM 生成结构化检索计划
- > 后端做约束校验、硬过滤、RAG 检索和分层排序
- > 系统主推一个具体商品，而不是甩出一堆搜索结果
- > Android 端流式展示文字、商品卡片和语音
- > 用户点开商品卡片后，在详情浮层继续追问或修正
- > 后端绑定当前 product_id，围绕当前商品上下文重新推荐
- > 用户可加入购物车、修改数量、模拟下单
- > 端到端评测框架反馈优化

1.2 为什么不能做成普通 RAG 商品搜索

普通做法：

用户：推荐防晒霜

系统：给 10 个防晒霜商品卡片

这种做法的问题是：

1. 只是把关键词搜索换成自然语言搜索，用户仍然要自己筛选。
2. 导购没有替用户做决策，仍然把选择压力留给用户。
3. 商品列表过多时，用户会继续纠结，不符合低压力交互目标。
4. 很难体现“理解商品属性、用户意图、个性化决策”。

本项目应该做：

用户：推荐防晒霜，但不要含酒精，也不要日系品牌

系统：

1. 解释理解到的真实意图：成分更稳、来源可控、降低踩雷风险
2. 明确说明已经排除含酒精和日系品牌
3. 主推一个最合适的具体商品

4. 保留 1-2 个备选，但不喧宾夺主
5. 提供低成本修正入口：更便宜、更清爽、更适合户外、不要这个品牌

这才是导购：系统先替用户做一个可解释的决策，再允许用户轻松修正。

1.3 项目成功的判断标准

不是“功能越多越好”，而是能否形成一个清晰的导购闭环：

理解需求 -> 给出主推产品 -> 解释为什么 -> 用户轻松修正 -> 重新推荐 -> 加购结算 -> 可评测优化

最终答辩时要强调：

- 我们不是商品搜索，而是低压力 AI 导购。
- 我们不是把 RAG 结果直接甩给用户，而是做约束驱动的决策。
- 我们不是让 LLM 自由编商品，而是由后端执行硬过滤和检索控制。
- 我们不是普通多轮聊天，而是以商品为锚点的多轮导购，避免上下文污染。
- 我们不是只做 Demo，而是有端到端评测和失败归因。

2. 用户体验设计：低压力、主推、可修正

2.1 低压力导购的核心原则

低压力不是“少问问题”，而是减少用户重复表达和反复比较的成本。

本项目交互原则：

1. **能推荐就先推荐**：不要上来问一堆澄清问题。
2. **先给明确主推**：不要把 10 个结果平铺给用户。
3. **解释用户背后的意图**：让用户知道系统为什么这么选。
4. **保留低成本修正入口**：用户不满意时，直接点或说一句就能调整。
5. **上下文围绕当前商品收敛**：用户在商品详情里追问时，只继承与该商品相关的约束。

2.2 文本回复的风格

回复需要专业、亲和、可解释，但不能油腻。

推荐写法：

我理解你更在意的是“成分更稳”和“品牌来源可控”，所以会先排除含酒精、日系品牌以及不适合敏感肌的防晒。剩
→ 下商品里，我优先给你选一款更适合日常通勤、质地清爽的。

不推荐写法：

亲爱的别担心，我太懂你啦，这个绝对适合你。

回复结构建议：

1. 一句话复述用户真实意图
2. 明确说明已处理的硬约束
3. 给出主推商品
4. 解释 2-3 个推荐证据
5. 给出可继续修正的入口

2.3 主推 1 个产品，而不是结果列表

推荐层级：

Primary Product: 1 个主推产品，重点展示

Alternatives: 1-2 个备选，弱化展示

Quick Actions: 允许用户快速修正需求

示例：

我先按“油皮、日常通勤、不含酒精、150 元以内”给你选一款。
主推这款「清爽通勤防晒乳」。它不含酒精，非日系品牌，质地偏清爽，价格也在你的预算内。

如果你想要更便宜、更防水，或者更适合三亚海边，我可以继续帮你换。

2.4 主动反问的边界

主动反问不能变成新的交互负担。

规则：

如果缺失的信息不影响推荐：先按默认场景推荐，再给修正入口。

如果缺失的信息会导致推荐不可靠：再主动追问。

例如：

- 用户说“推荐防晒霜”时，可以默认日常通勤，先给一款。
- 用户说“送长辈护肤品”但没有预算、性别、肤质时，可以先追问预算或送礼对象。

3. 功能范围与优先级

3.1 P0：必须完成

P0 是三周内必须完成的最小优秀闭环。

1. Android 原生 Kotlin + Jetpack Compose App
2. 聊天主界面
3. 文字输入与发送
4. WebSocket text_delta 流式回复
5. 主推商品卡片流式渲染
6. 商品详情 BottomSheet
7. 商品级追问 product_followup
8. 购物车增删改查
9. 商品数量编辑
10. 模拟下单
11. 后端 RAG 检索与硬约束过滤
12. 反选/排除约束：不要、不含、除了、排除
13. 基础评测脚本和 Demo 验收清单

3.2 P1：强烈建议完成

P1 是项目亮点，尽量完成。

1. Android 原生语音输入 ASR
2. TTS 句子级或片段级流式播放
3. 场景化组合推荐：三亚度假、通勤上班、学生开学等模板
4. bundle_start / bundle_item / bundle_done 组合卡片流式渲染
5. 多商品对比
6. 自然语言购物车操作
7. 个性化 session profile
8. 首 token / 首卡 / 首音频延迟统计

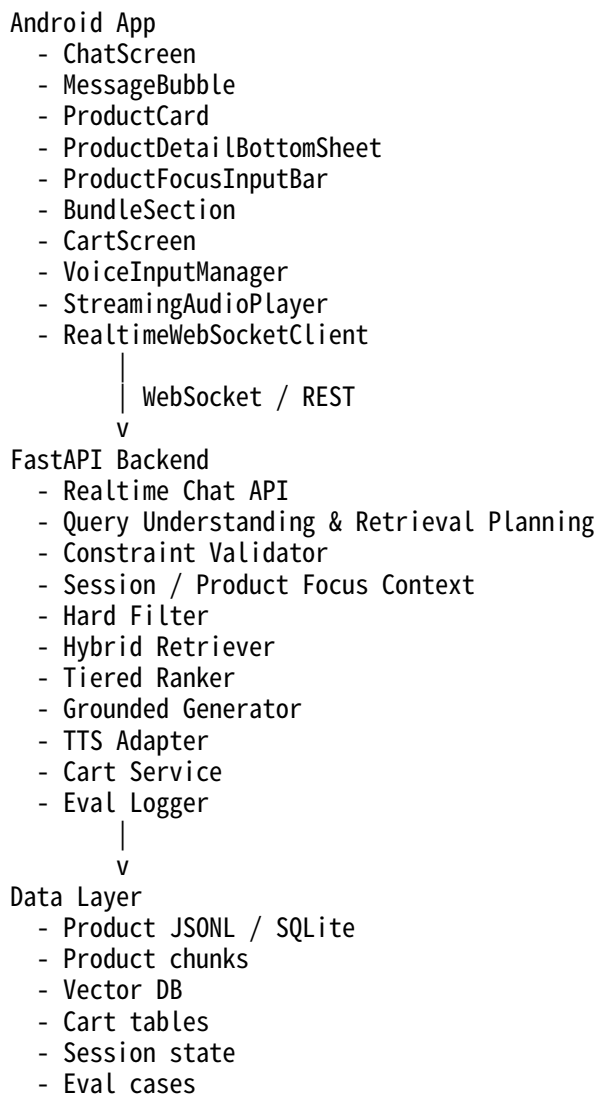
3.3 P2：有时间再做

1. 拍照找货
2. 订单历史
3. 复杂推荐模型训练
4. 真正登录注册

- 5. 真实支付
- 6. iOS 端
- 7. 复杂知识图谱推理

4. 总体系统架构

4.1 架构总览



4.2 两阶段 LLM + 确定性检索控制

LLM 负责理解语义，但不能直接决定最终商品。

本项目采用两阶段：

第一阶段 LLM：

负责意图识别、否定条件解析、场景理解、检索计划生成。

中间后端程序：

负责 JSON 校验、schema 对齐、metadata 硬过滤、RAG 检索、分层排序。

第二阶段 LLM：

只基于后端返回的真实候选商品做推荐解释和对话生成。

优势：

- 1. 利用 LLM 的自然语言理解能力。
- 2. 避免 LLM 自由编造商品。
- 3. 硬约束由后端程序确定性执行。
- 4. 排序逻辑可解释、可测试、可优化。

5. 商品数据与专属知识库

5.1 商品 Schema

商品必须从非结构化文案转成可决策对象。

示例：

```
{
  "product_id": "sku_101",
  "name": " 清爽通勤防晒乳",
  "category": " 防晒霜",
  "brand": " 示例品牌 B",
  "brand_region": " 中国",
  "price": 129.0,
  "stock": 120,
  "rating": 4.8,
  "main_image_url": "https://example.com/sku_101.jpg",
  "tags": [" 清爽", " 通勤", " 防晒", " 不含酒精"],
  "suitable_for": [" 油皮", " 混油皮", " 日常通勤"],
  "not_suitable_for": [" 长时间海边暴晒"],
  "ingredients": [" 烟酰胺", " 透明质酸钠"],
  "avoid_ingredients": [],
  "scenes": [" 通勤", " 日常防晒"],
  "selling_points": [" 清爽", " 不闷", " 性价比"],
  "description": " 适合油皮和混油皮日常通勤使用。",
  "reviews_summary": " 多数用户反馈质地清爽，成膜速度较快。"
}
```

关键字段：

字段	用途
category	类目硬过滤
brand / brand_region	品牌与地区排除
price	预算硬过滤和价格偏好
stock	缺货剔除
ingredients / avoid_ingredients	成分排除
suitable_for	人群和肤质匹配
scenes	场景匹配
selling_points	推荐解释证据
reviews_summary	评价摘要证据

5.2 知识库构建

每个商品构造一个主 chunk，不建议切得太碎。

商品名：清爽通勤防晒乳
类目：防晒霜
品牌：示例品牌 B
品牌地区：中国
价格：129 元

适合：油皮、混油皮、日常通勤

不适合：长时间海边暴晒

成分：烟酰胺、透明质酸钠

不含：酒精

场景：通勤、日常防晒

卖点：清爽、不闷、性价比

评价摘要：多数用户反馈质地清爽，成膜速度较快

向量库保存 chunk，数据库保存 metadata。检索时采用：

metadata 硬过滤 + 向量语义召回 + 可选关键词召回 + 分层重排

5.3 数据量建议

三周内建议准备 100-150 个商品，覆盖 5-6 个类目：

1. 洗面奶
2. 防晒霜
3. 面霜/精华
4. 蓝牙耳机
5. 跑鞋
6. 双肩包/旅行配件

每个类目要有：

1. 不同价格段
2. 不同适用人群
3. 不同使用场景
4. 不同品牌地区
5. 可被排除的属性
6. 适合对比的参数

6. 后端 RAG、检索计划与排序逻辑

6.1 LLM 生成结构化检索计划

用户输入不要直接送入向量库。第一步应由 LLM 生成结构化 plan。

输入：

推荐防晒霜，但我不要含酒精的，也不要日系品牌

LLM 输出：

```
{
  "intent": "recommend_product",
  "retrieval_mode": "single",
  "category": " 防晒霜",
  "hard_constraints": {
    "category": " 防晒霜",
    "exclude_ingredients": [" 酒精", " 乙醇"],
    "exclude_brand_regions": [" 日本"],
    "in_stock_only": true
  },
  "soft_preferences": {
    "texture": " 清爽",
    "scene": " 日常通勤"
  },
  "retrieval_query": " 清爽 通勤 防晒霜 不含酒精 非日系",
  "need_clarification": false
}
```


后端再执行：

1. JSON Schema 校验
2. 字段白名单校验
3. 类目标标准化
4. 成分同义词扩展
5. 品牌地区映射
6. session 上下文补全
7. 硬过滤
8. 检索与排序

6.2 反选/排除约束

必须支持：

不要含酒精
不含乙醇
不要日系品牌
除了某品牌
排除太贵的
不适合敏感肌的不要

原则：

must_not 是硬过滤，不是扣分。

违反硬约束的商品不得进入候选集，也不得出现在商品卡片中。

硬过滤伪代码：

```
def hard_filter(product, constraints):  
    if constraints.category and product.category != constraints.category:  
        return False  
    if constraints.price_max is not None and product.price > constraints.price_max:  
        return False  
    if constraints.in_stock_only and product.stock <= 0:  
        return False  
    for x in constraints.exclude_ingredients:  
        if x in product.ingredients or x in product.avoid_ingredients or x in product.tags:  
            return False  
    if product.brand_region in constraints.exclude_brand_regions:  
        return False  
    if product.brand in constraints.exclude_brands:  
        return False  
    return True
```

6.3 不采用固定线性加权排序

不建议使用：

```
final_score =  
    0.45 * semantic_score  
+ 0.25 * attribute_match_score  
+ 0.15 * personalization_score  
+ 0.10 * price_score  
+ 0.05 * popularity_score  
- hard_constraint_penalty
```

问题：

1. 各个分数尺度不一致。
2. 权重没有数据支撑。
3. 硬约束不应该扣分，而应该过滤。

4. 不同类目的排序重点不同。
5. 个性化不能覆盖用户当前明确约束。

正式采用：

Hard Filter

- > Eligibility Tier
- > In-tier Ranking
- > Grounded LLM Explanation

6.4 分层排序策略

Hard Filter

剔除不满足条件的商品：

类目不匹配
超预算
缺货
含禁忌成分
品牌/地区被排除
不适合明确人群

Eligibility Tier

候选商品分层：

Tier 1: 完全满足显式需求和主要软偏好
Tier 2: 满足核心需求，但部分软偏好弱匹配
Tier 3: 兜底候选，需要明确说明不完全匹配原因

In-tier Ranking

同层内按优先级排序：

1. 类目精确匹配
2. 显式属性覆盖度
3. 场景匹配度
4. 语义相关性
5. 个性化偏好
6. 价格友好度
7. 评分/热度

伪代码：

```
def rank_products(candidates, constraints, profile):  
    filtered = [p for p in candidates if hard_filter(p, constraints)]  
  
    def sort_key(p):  
        return (  
            exact_category_match(p, constraints),  
            attribute_coverage(p, constraints),  
            scene_match_score(p, constraints),  
            semantic_rank_score(p),  
            personalization_match(p, profile),  
            price_preference_score(p, constraints),  
            popularity_score(p)  
        )  
  
    return sorted(filtered, key=sort_key, reverse=True)
```

6.5 受控多跳 RAG

需要考虑多跳，但不要做开放式无限循环 Agent。

本项目采用受控多跳：

LLM 生成一次 retrieval_plan

后端校验 plan

后端按 plan 执行有限步检索

max_hops = 2 或 3

适用场景：

场景	检索模式
单品推荐	single
商品对比	state_then_detail
场景组合	decompose_parallel
商品级追问	product_focus_retrieval
购物车指代	state_then_action

商品对比示例：

用户：第一款和第三款哪个更适合油皮？

步骤：

1. 从 last_product_ids 找到 first/third
2. 读取两个商品详情
3. 检索评价摘要或参数 chunk
4. LLM 基于真实信息对比

6.6 场景化组合推荐

用户输入：

下周去三亚度假，帮我搭配一套从防晒到穿搭的方案

LLM 输出：

```
{
  "intent": "scenario_bundle_recommendation",
  "retrieval_mode": "decompose_parallel",
  "scenario": " 三亚度假",
  "scene_tags": [" 海边", " 高温", " 强紫外线", " 轻便", " 度假"],
  "bundle_slots": [
    {
      "slot": " 防晒霜",
      "category": " 防晒霜",
      "retrieval_query": " 三亚 海边 高倍 清爽 防水 防晒霜"
    },
    {
      "slot": " 晒后修复",
      "category": " 晒后修复",
      "retrieval_query": " 晒后修复 舒缓 补水 芦荟"
    },
    {
      "slot": " 遮阳帽",
      "category": " 遮阳帽",
      "retrieval_query": " 海边旅行 防晒 宽檐 轻便 遮阳帽"
    }
  ]
}
```

执行方式：

1. 每个 slot 独立检索。
2. 每个 slot 取 top1/top2。
3. 合并成 bundle。
4. Android 按分组流式展示商品。

7. 以商品为锚点的多轮导购

7.1 设计动机

用户购物时的核心压力包括：

不知道哪个真正适合自己

害怕踩雷

不想在很多结果里比较

对某个推荐不满意时不知道怎么自然修正

不想反复重复约束

因此，本项目采用“商品为锚点”的多轮导购，而不是普通搜索结果列表。

7.2 核心流程

用户普通提问

- > 系统给出主推商品
- > 用户点击商品卡片
- > 商品详情 BottomSheet 浮起
- > 用户围绕当前商品继续问
- > Android 发送 product_followup + focus_product_id
- > 后端在当前商品上下文内重新规划检索
- > 返回解释和替代商品

7.3 Product Focus Context

后端维护两层上下文：

Global Session Context:

用户长期偏好、预算、肤质、排除项、购物车

Product Focus Context:

当前正在讨论的 product_id

原始推荐约束

围绕该商品的追问

用户对该商品的不满意点

替代推荐历史

示例：

```
{
  "session_id": "demo_session_001",
  "global_profile": {
    "skin_type": "油皮",
    "budget_max": 150,
    "excluded_ingredients": ["酒精"],
    "excluded_brand_regions": ["日本"]
  },
  "active_focus": {
    "focus_type": "product",
    "product_id": "sku_101",
```

```

    "product_name": " 高倍清爽防晒乳",
    "origin_constraints": {
      "category": " 防晒霜",
      "exclude_ingredients": [" 酒精"],
      "exclude_brand_regions": [" 日本"]
    },
    "local_constraints": {
      "price_max": 100,
      "prefer_texture": " 更清爽"
    },
    "followup_history": [
      " 这个有点贵, 有没有 100 以内的? "
    ]
  }
}

```

7.4 商品级追问示例

用户:

这个有点贵, 有没有 100 以内的?

后端理解:

当前商品 = sku_101

用户不满意点 = 价格太高

保留原始约束 = 防晒霜、不含酒精、非日系品牌

新增约束 = price <= 100

重新检索替代商品

Android 发送:

```

{
  "type": "product_followup",
  "session_id": "demo_session_001",
  "focus_product_id": "sku_101",
  "message": " 这个有点贵, 有没有 100 以内的? ",
  "tts_enabled": true
}

```

后端返回:

我保留了“不含酒精、非日系品牌”的要求, 并把预算压到 100 元以内重新筛选。更推荐这款清爽通勤防晒乳。

再返回 replacement_product。

8. Android 客户端整体设计

8.1 Android 端职责

Android 端不负责 RAG、排序和 LLM 推理。它负责体验呈现与交互闭环:

1. 聊天页面
2. 文字输入
3. WebSocket 流式通信
4. AI 文字逐字渲染
5. 商品卡片实时渐进式渲染
6. 商品详情 BottomSheet
7. 商品级追问输入框
8. 场景组合方案分组展示
9. TTS 音频流播放

10. 语音输入 ASR
11. 购物车页面
12. 购物车增删改查
13. 商品数量编辑
14. 模拟下单确认

8.2 推荐技术栈

Language: Kotlin
UI: Jetpack Compose
Architecture: MVVM
State: StateFlow
Network: OkHttp WebSocket + REST
Image: Coil
Local storage: DataStore 或 SharedPreferences
ASR: Android SpeechRecognizer
TTS playback: AudioTrack
Build: Gradle Wrapper

8.3 Android 工程目录

```
app/src/main/java/com/example/shopguideagent/  
  MainActivity.kt  
  config/  
    AppConfig.kt  
    UserSession.kt  
  data/  
    model/  
      ChatMessage.kt  
      Product.kt  
      Cart.kt  
      BundleModels.kt  
      RealtimeEvent.kt  
    remote/  
      RealtimeChatWebSocketClient.kt  
      ProductApiClient.kt  
      CartApiClient.kt  
    local/  
      SessionStore.kt  
  vm/  
    ChatViewModel.kt  
    CartViewModel.kt  
    ProductDetailViewModel.kt  
  ui/  
    screen/  
      ChatScreen.kt  
      CartScreen.kt  
    component/  
      MessageBubble.kt  
      ChatInputBar.kt  
      ProductCard.kt  
      ProductSkeletonCard.kt  
      ProductCarousel.kt  
      ProductDetailBottomSheet.kt  
      ProductFocusInputBar.kt  
      BundleSection.kt  
      CartItemCard.kt  
      CartSummaryBar.kt
```

```
CheckoutBottomSheet.kt
voice/
  VoiceInputManager.kt
audio/
  StreamingAudioPlayer.kt
navigation/
  AppNavGraph.kt
```

8.4 可借鉴的经典 Android 项目

不要从 0 硬写，但也不要复制整个项目。建议借鉴结构和组件思路：

参考项目	借鉴点
Now in Android	Compose 工程结构、Navigation、Design System、UI state
Android Compose Samples / Jetchat	聊天气泡、输入栏、消息列表
Android Architecture Samples	ViewModel、Repository、Flow、单 Activity 架构
Sunflower	详情页、图片内容布局、Material 风格
Pokedex Compose	商品卡片、详情页、图片加载、动效
Stream Chat Android / AI Chat UI	AI 聊天交互、typing indicator、富消息思路
AndroidX Media / UAMP	音频播放状态与生命周期管理

借鉴原则：

Jetchat -> 聊天气泡和输入区
Now in Android -> 工程分层和 UI state
Architecture Samples -> ViewModel + Repository + Flow
Sunflower / Pokedex Compose -> 商品卡片和详情页
Stream Chat -> AI 聊天交互细节
Media3 / UAMP -> 音频播放生命周期思路

9. Android 核心页面与组件

9.1 ChatScreen

承载主流程：

TopBar

- 标题：智能导购助手
- 副标题：懂你需求的 AI 购物助手
- 购物车图标 + badge

LazyColumn

- 用户气泡
- AI 气泡
- 主推商品卡片
- 备选商品卡片
- 场景组合 BundleSection

Bottom Input

- 文本输入框
- 麦克风按钮
- 发送按钮

9.2 商品卡片流式渲染

事件流程：

products_start -> 显示 skeleton 区域
product_item -> 对应卡片淡入
products_done -> 停止 loading, 激活按钮

ProductCard 内容：

商品图
商品名
价格
标签
一句推荐理由
查看详情 / 加入购物车

视觉建议：

主推卡片更大、更突出
备选卡片弱化
卡片圆角 20dp
轻阴影
图片失败要 fallback
不要一次性平铺太多商品

9.3 ProductDetailBottomSheet

这是核心交互创新，不建议做普通页面跳转。

结构：

商品主图
商品名 / 价格 / 标签
为什么推荐给你
- 符合“不含酒精”
- 非日系品牌
- 适合油皮通勤
关键属性
可能不适合的情况
加入购物车按钮
Quick Actions
- 换个更便宜的
- 换个更清爽的
- 不要这个品牌
- 更适合户外
ProductFocusInputBar
- 针对这款商品继续问...
替代商品区域

9.4 BundleSection

用于场景化组合推荐。

三亚度假组合方案

[防晒护理]

防晒霜卡片
晒后修复卡片

[穿搭]

轻薄外套卡片

凉鞋卡片

[出行配件]

遮阳帽卡片

墨镜卡片

防水包卡片

[一键加入购物车]

9.5 CartScreen

购物车能力：

1. 添加商品
2. 删除商品
3. 修改数量
4. 全选/取消
5. 合计金额
6. 清空购物车
7. 模拟下单

CartItemCard：

选择框

商品图

商品名

标签

单价

数量编辑器 [-] quantity [+]

删除按钮

10. Android 与后端协议

10.1 客户端发给后端

user_message

```
{
  "type": "user_message",
  "session_id": "demo_session_001",
  "message": " 推荐一款适合油皮的洗面奶，预算 100 以内",
  "input_type": "text",
  "tts_enabled": true,
  "voice": "default_female"
}
```

product_followup

```
{
  "type": "product_followup",
  "session_id": "demo_session_001",
  "focus_product_id": "sku_101",
  "message": " 这个有点贵，有没有 100 以内的？ ",
  "tts_enabled": true
}
```

cart_action

```
{
```

```

    "type": "cart_action",
    "session_id": "demo_session_001",
    "action": "add_to_cart",
    "product_id": "sku_101",
    "quantity": 1
}

```

10.2 后端发给 Android

text_delta

```

{
  "type": "text_delta",
  "message_id": "assistant_001",
  "text": " 我"
}

```

product_item

```

{
  "type": "product_item",
  "message_id": "assistant_001",
  "index": 0,
  "role": "primary",
  "product": {
    "product_id": "sku_101",
    "name": " 清爽通勤防晒乳",
    "price": 129,
    "main_image_url": "https://example.com/sku_101.jpg",
    "tags": [" 不含酒精", " 清爽", " 通勤"],
    "reason": " 适合油皮日常通勤, 且符合不含酒精和非日系品牌要求"
  }
}

```

replacement_product

```

{
  "type": "replacement_product",
  "focus_product_id": "sku_101",
  "reason": " 更便宜, 但仍满足不含酒精和非日系品牌",
  "product": {
    "product_id": "sku_118",
    "name": " 清爽通勤防晒乳平价款",
    "price": 89,
    "tags": [" 不含酒精", " 清爽", " 通勤"],
    "reason": " 价格低于 100, 保留原有排除约束"
  }
}

```

bundle_item

```

{
  "type": "bundle_item",
  "message_id": "assistant_001",
  "bundle_id": "bundle_001",
  "group": " 防晒护理",
  "slot": " 防晒霜",
  "index": 0,
}

```

```

"product": {
  "product_id": "sku_201",
  "name": " 高倍清爽防水防晒乳",
  "price": 139,
  "tags": ["SPF50", " 防水", " 海边"],
  "reason": " 适合三亚强紫外线和海边活动"
}
}

```

audio_delta

```

{
  "type": "audio_delta",
  "message_id": "assistant_001",
  "segment_id": 1,
  "audio_format": "pcm_s16le",
  "sample_rate": 24000,
  "channels": 1,
  "audio_base64": "...
}

```

10.3 REST 接口

```

GET /api/cart?session_id=xxx
POST /api/cart/add
POST /api/cart/add_bundle
POST /api/cart/update_quantity
POST /api/cart/remove
POST /api/cart/select
POST /api/cart/clear
POST /api/cart/checkout

```

11. Android Studio 与 Codex / Claude Code 协作

11.1 推荐协作模式

Android Studio:

创建工程、Gradle Sync、真机运行、UI 预览、Logcat 调试。

Codex / Claude Code:

批量写 Kotlin/Compose 代码、补组件、修编译错误、重构、跑 gradlew。

Git:

保存每个阶段的稳定版本。

不要二选一。Android Studio 是专用 IDE，Codex / Claude Code 是代码执行代理。

11.2 每次给 Codex 的固定模板

请先阅读：

1. AGENTS.md
2. prompts/03_TASK_CHAT_UI.md

只实现 03_TASK_CHAT_UI.md 中要求的内容。

不要实现后续任务，不要做无关重构。

完成后请：

1. 运行 `./gradlew :app:assembleDebug`

- 2. 修复所有编译错误
- 3. 总结修改了哪些文件
- 4. 说明还有哪些未完成

11.3 任务顺序

01_ANDROID_STUDIO_CODEX_WORKFLOW.md
02_TASK_PROJECT_SCAFFOLD.md
03_TASK_CHAT_UI.md
04_TASK_WEBSOCKET_STREAMING.md
05_TASK_PRODUCT_CARDS_AND_BUNDLE.md
06_TASK_CART_CRUD.md
07_TASK_VOICE_ASR_AND_STREAMING_TTS.md
08_TASK_TESTING_AND_DEMO_ACCEPTANCE.md
09_TASK_PRODUCT_FOCUS_GUIDANCE.md
10_TASK_BACKEND_CONTRACT_AND MOCK_SERVER.md

11.4 不要让 Codex 一次做完全部

错误：
请你把整个 Android 客户端都实现出来。
正确：
请只完成 Task 03 聊天 UI，不要实现 WebSocket、购物车和 TTS。

12. 三周实施计划与分工

12.1 Week 1：端到端最小闭环

目标：文本推荐 + 商品卡片 + 购物车基础。

天数	后端	Android
Day 1	定 schema 和协议	创建 Compose 工程
Day 2	商品数据整理	ChatScreen 静态 UI
Day 3	SQLite + 向量库	MessageBubble + ProductCard mock
Day 4	检索与硬过滤	ProductDetailBottomSheet mock
Day 5	LLM retrieval_plan	WebSocket text_delta
Day 6	product_item 协议	商品卡片流式渲染
Day 7	联调推荐流程	真机测试和 UI 修正

12.2 Week 2：差异化能力

目标：商品级追问、购物车、场景组合、语音/TTS。

天数	后端	Android
Day 8	product_focus_context	商品详情输入框
Day 9	product_followup	replacement_product 渲染
Day 10	cart API	CartScreen 数量编辑
Day 11	bundle 检索	BundleSection
Day 12	TTS stream	AudioTrack 播放
Day 13	ASR 协议	SpeechRecognizer
Day 14	联调全链路	Demo UI 打磨

12.3 Week 3：评测、稳定、答辩

天数	任务
Day 15	构造 eval_cases.jsonl
Day 16	修复反选、约束、幻觉失败样本
Day 17	UI 精修和流式体验优化
Day 18	首 token / 首卡 / 首音频延迟优化
Day 19	README、架构文档、API 文档
Day 20	录 Demo 视频，准备答辩材料
Day 21	彩排，修最后 bug

12.4 两人分工

后端/RAG 同学：

商品 schema
数据清洗
向量库
retrieval_plan
constraint validator
hard filter
tiered ranking
product_focus_context
bundle 检索
cart API
eval runner

Android 同学：

Compose 工程
ChatScreen
ProductCard
ProductDetailBottomSheet
ProductFocusInputBar
BundleSection
CartScreen
WebSocket client
SpeechRecognizer
AudioTrack
UI 动效和 Demo 录屏

13. 评测、验收与答辩

13.1 必测 Query

1. 推荐一款适合油皮的洗面奶，预算 100 以内
2. 推荐防晒霜，但不要含酒精的，也不要日系品牌
3. 下周去三亚度假，帮我搭配一套从防晒到穿搭的方案
4. 点开主推商品，输入：这个有点贵，有没有 100 以内的？
5. 点开主推商品，输入：这个适合敏感肌吗？
6. 把第一款加入购物车，把数量改成 2
7. 看看购物车，然后下单

13.2 评测指标

模块	指标
意图理解	intent accuracy
约束抽取	constraint extraction accuracy
反选约束	must_not violation rate
主推商品	primary recommendation satisfaction
商品追问	product_followup success rate
替代商品	replacement constraint inheritance accuracy
场景组合	required slot coverage
购物车	cart operation accuracy
流式体验	first token / first card / first audio latency
稳定性	crash-free demo rate

13.3 Android 验收清单

1. App 可安装。
2. 聊天页不白屏。
3. text_delta 流式显示。
4. 商品卡片 skeleton 后逐张出现。
5. 主推商品视觉突出。
6. 商品详情浮层可打开。
7. 商品级追问可发送。
8. 替代商品可显示。
9. bundle 可分组显示。
10. 购物车可改数量和下单。
11. 语音输入可用。
12. TTS 可播放并停止。
13. 后端错误不崩溃。
14. 图片加载失败有 fallback。

13.4 答辩表达

可以这样讲：

我们不是做一个自然语言商品搜索，而是做低压力 AI 导购。
 系统会先理解用户背后的购买动机和约束，给出一个明确主推商品。
 当用户对主推商品不满意时，可以直接在商品详情浮层继续追问。
 后端把追问绑定到当前 product_id，并继承原始约束，重新检索替代商品。
 这样减少了用户重复输入，也避免了多轮上下文污染。

14. 风险控制

14.1 风险：只推一个商品不稳

解决：前端突出主推 1 个，但后端保留 top3。必要时展示“换一款”和“看备选”。

14.2 风险：LLM 漏掉否定约束

解决：LLM 输出后必须经过 constraint validator；否定词用规则和同义词表二次校验。

14.3 风险：上下文污染

解决：商品详情输入必须带 focus_product_id；后端只继承当前商品相关约束；跨类目时清空不相关约束。

14.4 风险：TTS 卡顿

解决：TTS 按句子或短片段合成；Android 端先缓存 200-500ms；TTS 失败不影响文字和商品卡片。

14.5 风险：Codex 改太多导致项目失控

解决：每次只给一个任务 MD；每个任务必须 `./gradlew :app:assembleDebug` 通过；每完成一项就 git commit。

15. 最终交付物

1. Android APK
2. FastAPI 后端
3. 商品数据 `products.jsonl`
4. 向量库构建脚本
5. WebSocket / REST 协议文档
6. `eval_cases.jsonl`
7. 评测脚本与报告
8. README
9. AGENTS.md
10. Codex / Claude Code 任务 prompts
11. Demo 视频
12. 答辩 PPT

附录 A：核心事件列表

客户端 -> 后端：

`user_message`
`product_followup`
`cart_action`

后端 -> 客户端：

`text_delta`
`products_start`
`product_item`
`products_done`
`bundle_start`
`bundle_item`
`bundle_done`
`focus_text_delta`
`replacement_product`
`focus_done`
`audio_delta`
`cart_update`
`done`
`error`

附录 B：最小 Demo 脚本

步骤 1：用户语音或文字输入

推荐防晒霜，但不要含酒精的，也不要日系品牌

步骤 2：系统流式回复

解释已理解用户更在意成分安全和品牌来源可控。

步骤 3：主推商品卡片出现

主推 1 个商品，显示价格、标签、推荐理由。

步骤 4：用户点击商品卡片
ProductDetailBottomSheet 浮出，不跳转页面。

步骤 5：用户在详情浮层输入
这个有点贵，有没有 100 以内的？

步骤 6：系统返回替代商品
保留原始约束，并新增 price <= 100。

步骤 7：加入购物车
购物车 badge 更新。

步骤 8：进入购物车
修改数量为 2，模拟下单。

附录 C：最终核心卖点

1. 低压力导购，不是搜索结果列表。
2. 主推一个具体商品，减少用户选择压力。
3. 以商品为锚点支持多轮追问和替代推荐。
4. LLM 只生成检索计划，后端确定性执行硬过滤和排序。
5. 反选/排除约束作为硬过滤，避免约束违反。
6. 场景化组合推荐支持跨类目 slot 并行检索。
7. Android 端实现文字、商品卡片、组合方案、TTS 的流式体验。
8. 购物车 CRUD 和模拟下单形成交易闭环。
9. 端到端评测框架支持反馈优化。